



JERUSALEM COLLEGE OF ENGINEERING

AN AUTONOMOUS INSTITUTION

Approved by AICTE, New Delhi, Affiliated to Anna University Chennai,
Accredited by NBA, New Delhi and Accredited by NAAC with "A" Grade.
Velachery Main Road, Narayanapuram, Pallikaranai, Chennai, Tamil Nadu - 600100.

UNIT-IV

KNOWLEDGE AND REASONING

First-Order Logic – Knowledge Representation - Syntax and Semantics of First-Order Logic - Using First-Order Logic - Knowledge Engineering in First-Order Logic - Inference in First-Order Logic - Propositional vs. First-Order Inference - Unification and First-Order Inference - Forward Chaining - Backward Chaining - Quantifying Uncertainty - Bayes' Rule and Its Use - Naive Bayes Model.

First-Order Logic

First-order logic (FOL), also known as predicate logic or first-order predicate calculus, is a powerful framework used in various fields such as mathematics, philosophy, linguistics, and computer science. In artificial intelligence (AI), FOL plays a crucial role in knowledge representation, automated reasoning, and natural language processing.

Fundamentals of First-Order Logic

First-order logic extends **propositional logic** by incorporating quantifiers and predicates, allowing for more expressive statements about the world. The key components of FOL include constants, variables, predicates, functions, quantifiers, and logical connectives.

1. **Constants:** Constants represent specific objects within the domain of discourse. For example, in a given domain, Alice, 2, and NewYork could be constants.
2. **Variables:** Variables stand for unspecified objects in the domain. Commonly used symbols for variables include x , y , and z .
3. **Predicates:** Predicates are functions that return true or false, representing properties of objects or relationships between them. For example, Likes(Alice, Bob) indicates that Alice likes Bob, and GreaterThan(x , 2) means that x is greater than 2.
4. **Functions:** Functions map objects to other objects. For instance, MotherOf(x) might denote the mother of x .
5. **Quantifiers:** Quantifiers specify the scope of variables. The two main quantifiers are:
 - **Universal Quantifier ($\forall$):** Indicates that a predicate applies to all elements in the domain.
 - For example, $\forall x (\text{Person}(x) \rightarrow \text{Mortal}(x))$ means "All persons are mortal."
 - **Existential Quantifier ($\exists$):** Indicates that there is at least one element in the domain for which the predicate holds.
 - For example, $\exists x (\text{Person}(x) \wedge \text{Likes}(x, \text{IceCream}))$ means "There exists a person who likes ice cream."
6. **Logical Connectives:** Logical connectives include conjunction ($\wedge$), disjunction ($\vee$), implication ($\rightarrow$), biconditional ($\leftrightarrow$), and negation ($\neg$). These connectives are used to form complex logical statements.

Syntax and Semantics of First-Order Logic

The syntax of first-order logic specifies the rules for constructing valid expressions, including terms and formulas. Terms are expressions that refer to objects and include constants, variables, and functions. Formulas are logical statements formed by combining predicates, terms, quantifiers, and logical connectives.

The semantics of FOL define the meaning of the expressions. An interpretation assigns a domain of discourse and provides meaning to the constants, functions, and predicates. A formula is considered true under an interpretation if it accurately describes the relationships and properties of the objects in the domain.

For instance, in the domain of natural numbers, let $\text{GreaterThan}(x, y)$ be a predicate that holds if x is greater than y . If x is assigned the value 5 and y is assigned the value 3, then $\text{GreaterThan}(5, 3)$ would be true under this interpretation.

Applications of First-Order Logic in AI

First-order logic is instrumental in various AI applications, enabling systems to reason about complex knowledge structures and make informed decisions. Some of the key applications of FOL in AI include:

1. **Knowledge Representation:** FOL provides a robust framework for representing complex relationships and properties of objects. For example, in a medical diagnosis system, predicates can represent symptoms, diseases, and their relationships, allowing the system to reason about potential diagnoses.
2. **Automated Theorem Proving:** Automated theorem proving involves using algorithms to prove mathematical theorems. FOL provides the foundational structure for many theorem proving systems, enabling them to verify software and hardware correctness, prove mathematical theorems, and more.
3. **Natural Language Processing (NLP):** In NLP, FOL is used to parse and understand natural language by representing the meaning of sentences in a formal, logical manner. This allows AI systems to perform tasks such as question answering, machine translation, and text summarization.
4. **Expert Systems:** Expert systems encode expert knowledge using FOL and reason about it to make decisions. For example, a legal expert system might use FOL to represent legal rules and infer outcomes based on given facts, aiding in legal decision-making processes.
5. **Semantic Web:** The Semantic Web uses FOL to describe relationships between web resources, enabling more intelligent information retrieval and processing. This enhances the ability of AI systems to understand and interact with web content.

Example: Using First-Order Logic for Reasoning

To illustrate the use of FOL in reasoning, consider the following knowledge base:

1. $\forall x (\text{Cat}(x) \rightarrow \text{Mammal}(x))$ (All cats are mammals)
2. $\forall x (\text{Mammal}(x) \rightarrow \text{Animal}(x))$ (All mammals are animals)
3. $\text{Cat}(\text{Tom})$ (Tom is a cat)

From these statements, we can infer:

1. $\text{Mammal}(\text{Tom})$ (Since Tom is a cat, and all cats are mammals)
2. $\text{Animal}(\text{Tom})$ (Since Tom is a mammal, and all mammals are animals)

These inferences demonstrate how FOL can be used to derive new knowledge from existing facts. The ability to reason logically about relationships and properties is a key strength of FOL in AI.

Advanced Concepts in First-Order Logic

1. **Unification:** Unification is the process of finding a substitution that makes two logical expressions identical. It is a fundamental operation in automated reasoning and logic programming, enabling the matching of predicates with variables.
2. **Resolution:** Resolution is a rule of inference used for automated theorem proving. It involves refuting a set of clauses by deriving a contradiction, proving that the original statement is true. Resolution is a powerful method for proving logical theorems in FOL.
3. **Model Checking:** Model checking is a technique used to verify the correctness of systems with respect to a given specification. FOL is used to express the properties and behaviors of the system, enabling the verification of complex systems such as software, hardware, and protocols.
4. **Logic Programming:** Logic programming languages, such as Prolog, use FOL to express programs. These languages allow for declarative programming, where the programmer specifies what needs to be done rather than how to do it. Logic programming is used in various AI applications, including natural language processing, expert systems, and knowledge representation.

Challenges and Limitations

Despite its power and expressiveness, FOL has some limitations and challenges:

1. **Computational Complexity:** Reasoning in FOL can be computationally expensive, particularly for large knowledge bases. Finding efficient algorithms for reasoning in FOL is an ongoing research area in AI.
2. **Expressiveness vs. Decidability:** While FOL is more expressive than propositional logic, it is also undecidable, meaning there is no general algorithm that can determine the truth of all FOL statements. This trade-off between expressiveness and decidability is a key consideration in AI.
3. **Handling Uncertainty:** FOL is not well-suited for handling uncertainty and probabilistic reasoning, which are common in real-world AI applications. Extensions to FOL, such as probabilistic logic and fuzzy logic, have been developed to address these limitations.

Knowledge Engineering in First-Order Logic

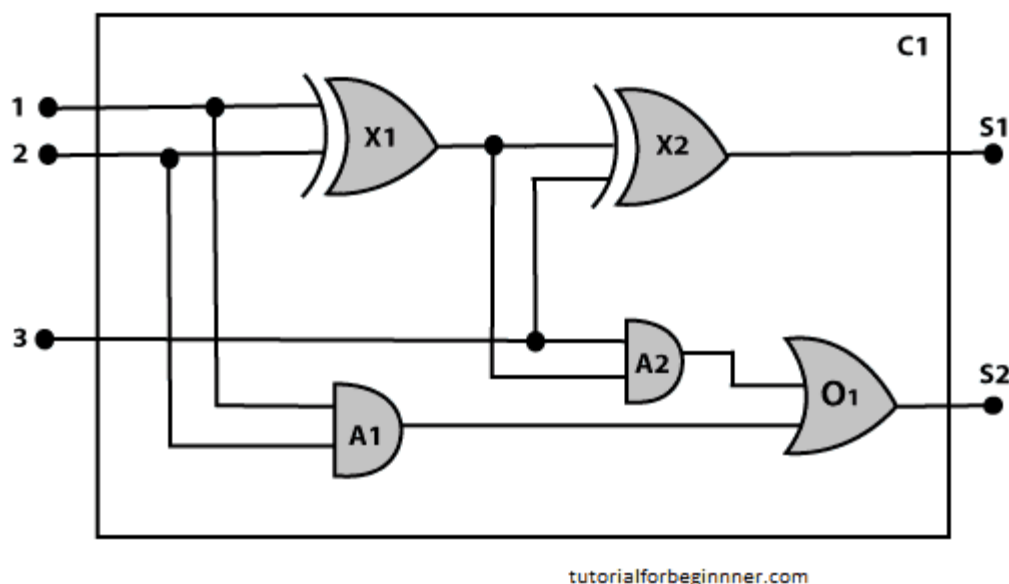
What is knowledge-engineering?

Knowledge engineering is the process of building a knowledge base in first-order logic. In knowledge engineering, a knowledge engineer is someone who researches a given domain, learns significant domain concepts, and develops a formal representation of the objects.

In this topic, we'll look at the knowledge engineering process in the context of an electronic circuit, which we're already familiar with. This method is best suited for developing **specialized knowledge bases**.

The knowledge-engineering process:

The knowledge-engineering process is broken down into the following steps. We will construct a knowledge foundation that will allow us to reason about the digital circuit (One-bit full adder) that is shown below using these techniques.



1. Identify the task:

The initial stage in the procedure is to determine the task, and there are several reasoning tasks for the digital circuit.

We'll look at the circuit's functionality at the highest level, or first level:

- Does the circuit add properly?
- What will be the output of gate A2, if all the inputs are high?

At the second level, we will inspect the circuit structure details such as:

- Which gate is connected to the first input terminal?
- Does the circuit have feedback loops?

2. Assemble the relevant knowledge:

In the second step, we will put together the necessary information for digital circuits. As a result, we need to know the following things about digital circuits:

- Wires and gates are used in logic circuits.
- The signal travels through wires to the gate's input terminal, and each gate produces an output that travels farther.
- There are four types of gates in this logic circuit: **AND, OR, XOR, and NOT**.
- There is one output terminal and two input terminals on each of these gates (except NOT gate, it has one input terminal).

3. Decide on vocabulary:

Selecting functions, predicates, and constants to represent circuits, terminals, signals, and gates is the next phase in the process. To begin, we'll separate the gates from one another and from other items. Each gate is represented as an object with a unique name, such as **Gate(X1)**. Each gate's functionality is determined by its type, which is represented by constants such as **AND, OR, XOR, and NOT**. A predicate will be used to identify circuits: **Circuit(C1)**.

For the terminal, we will use predicate: **Terminal(x)**.

The function **In(1, X1)** will be used to denote the first input terminal of the gate, and **Out(1, X1)** will be used to denote the output terminal.

The connectivity between gates can be represented by predicate **Connect(Out(1, X1), In(1, X1))**.

We use a unary predicate **On(t)**, which is true if the signal at a terminal is on.

4. Encode general knowledge about the domain:

We'll need the following principles to encode general information about logic circuits:

- When two terminals are connected, the input signal is the same, which can be represented as:

$$\forall t1, t2 \text{ Terminal}(t1) \wedge \text{Terminal}(t2) \wedge \text{Connect}(t1, t2) \rightarrow \text{Signal}(t1) = \text{Signal}(t2).$$

- Every terminal's signal will have a value of 0 or 1, and will be represented as:

$$\forall t \text{ Terminal}(t) \rightarrow \text{Signal}(t) = 1 \vee \text{Signal}(t) = 0.$$

- Connect predicates are commutative:

$$\forall t1, t2 \text{ Connect}(t1, t2) \rightarrow \text{Connect}(t2, t1).$$

- Representation of types of gates:

$$g \text{ Gate}(g) \wedge r = \text{Type}(g) \rightarrow r = \text{OR} \vee r = \text{AND} \vee r = \text{XOR} \vee r = \text{NOT}.$$

- If and only if any of the AND gate's inputs are 0, the output will be zero.

$$g \text{ Gate}(g) \wedge \text{Type}(g) = \text{AND} \rightarrow \text{Signal}(\text{Out}(1, g)) = 0 \Leftrightarrow \exists n \text{ Signal}(\text{In}(n, g)) = 0.$$

- Output of OR gate is 1 if and only if any of its input is 1:

$$\forall g \text{ Gate}(g) \wedge \text{Type}(g) = \text{OR} \rightarrow \text{Signal}(\text{Out}(1, g)) = 1 \Leftrightarrow \exists n \text{ Signal}(\text{In}(n, g)) = 1$$

- Output of XOR gate is 1 if and only if its inputs are different:

$$\forall g \text{ Gate}(g) \wedge \text{Type}(g) = \text{XOR} \rightarrow \text{Signal}(\text{Out}(1, g)) = 1 \Leftrightarrow \text{Signal}(\text{In}(1, g)) \neq \text{Signal}(\text{In}(2, g)).$$

- Output of NOT gate is invert of its input:

$$\forall g \text{ Gate}(g) \wedge \text{Type}(g) = \text{NOT} \rightarrow \text{Signal}(\text{In}(1, g)) \neq \text{Signal}(\text{Out}(1, g)).$$

- The gates in the circuit above all have two inputs and one output (except NOT gate).

$$\begin{aligned} \forall g \text{ Gate}(g) \wedge \text{Type}(g) = \text{NOT} &\rightarrow \text{Arity}(g, 1, 1) \\ \forall g \text{ Gate}(g) \wedge r = \text{Type}(g) \wedge (r = \text{AND} \vee r = \text{OR} \vee r = \text{XOR}) &\rightarrow \text{Arity}(g, 2, 1). \end{aligned}$$

- All gates are logic circuits:

$$\forall g \text{ Gate}(g) \rightarrow \text{Circuit}(g).$$

5. Encode a description of the problem instance:

Now, in order to decode the problem of circuit C1, we must first classify the circuit and its gate components. This phase is simple if the problem's ontology has already been considered. This step entails writing simple atomic sentences of idea instances, which is referred to as ontology.

We may encode the issue instance in atomic phrases for the given circuit C1 as follows:

Because there are two XOR gates, two AND gates, and one OR gate in the circuit, the atomic sentences for these gates will be:

For XOR gate: $Type(x1) = XOR, Type(X2) = XOR$
For AND gate: $Type(A1) = AND, Type(A2) = AND$
For OR gate: $Type(O1) = OR.$

And then represent the connections between all the gates.

Note: Ontology defines a particular theory of the nature of existence.

6. Pose queries to the inference procedure and get answers:

We'll find all of the possible values for all of the terminals in the adder circuit in this stage. The initial question will be:

What input combination should result in the first output of circuit C1 being 0 and the second output being 1?

$\exists i1, i2, i3 \text{ Signal (In(1, C1))} = i1 \wedge \text{Signal (In(2, C1))} = i2 \wedge \text{Signal (In(3, C1))} = i3$
 $\wedge \text{Signal (Out(1, C1))} = 0 \wedge \text{Signal (Out(2, C1))} = 1$

7. Debug the knowledge base:

We'll now debug the knowledge base, which is the final step in the process. In this stage, we'll try to figure out what's wrong with the knowledge base.

We may have missed statements like $1 \neq 0$ in the knowledge base.

Inference in first order logic

In First-Order Logic, inference is used to derive new facts or sentences from existing ones. Before we get into the FOL inference rule, it's important to understand some basic FOL terminology.

Substitution:

Substitution is a basic procedure that is applied to terms and formulations. It can be found in all first-order logic inference systems. When there are quantifiers in FOL, the substitution becomes more complicated. When we write $F[a/x]$, we are referring to the substitution of a constant "a" for the variable "x."

[**Note: first-order logic can convey facts about some or all of the universe's objects.**]

Equality:

In First-Order Logic, atomic sentences are formed not only via the use of predicate and words, but also through the application of equality. We can do this by using **equality symbols**, which indicate that the two terms relate to the same thing.

Example: **Brother** **(John)** = **Smith.**

In the above example, the object referred by the **Brother (John)** is close to the object referred by **Smith**. The equality symbol can be used with negation to portray that two terms are not the same objects.

Example: $\neg(x=y)$ which is equivalent to $x \neq y$.

FOL inference rules for quantifier:

First-order logic has inference rules similar to propositional logic, therefore here are some basic inference rules in FOL:

- **Universal Generalization**
- **Universal Instantiation**
- **Existential Instantiation**
- **Existential introduction**

1. Universal Generalization:

- Universal generalization is a valid inference rule that states that if premise $P(c)$ is true for any arbitrary element c in the universe of discourse, we can arrive at the conclusion $\forall x P(x)$.
- It can be represented as:
$$\frac{P(c)}{\forall x P(x)}$$
- If we want to prove that every element has a similar property, we can apply this rule.
- x must not be used as a free variable in this rule.

Example: Let's represent, $P(c)$: "A byte contains 8 bits", so "All bytes contain 8 bits." for $\forall x P(x)$, it will also be true.

2. Universal Instantiation:

- A valid inference rule is universal instantiation, often known as universal elimination or UI. It can be used to add additional sentences many times.
- The new knowledge base is logically equal to the existing knowledge base.
- **We can infer any phrase by replacing a ground word for the variable**, according to UI
- The UI rule says that we can infer any sentence $P(c)$ by substituting a ground term c (a constant within domain x) from $\forall x P(x)$ for any object in the universe of discourse.
- It can be represented as

$$\frac{P(c)}{\forall x P(x)}$$

Example: 1 IF "Every person like ice-cream" $\Rightarrow \forall x P(x)$ so we can infer that
"John likes ice-cream" $\Rightarrow P(c)$

Example: 2 Let's take a famous example, "All kings who are greedy are Evil." So let our knowledge base contains this detail as in the form of FOL: $\forall x \text{king}(x) \wedge \text{greedy}(x) \rightarrow \text{Evil}(x)$, We can infer any of the following statements using Universal Instantiation from this information:

- **King(John) $\wedge$ Greedy (John) $\rightarrow$ Evil (John),**
- **King(Richard) $\wedge$ Greedy (Richard) $\rightarrow$ Evil (Richard),**
- **We can infer any phrase by replacing a ground word for the variable, according to UI**
- **King(Father(John)) $\wedge$ Greedy (Father(John)) $\rightarrow$ Evil (Father(John)),**

3. Existential Instantiation:

- Existential instantiation is also known as Existential Elimination, and it is a legitimate first-order logic inference rule.
- It can only be used to replace the existential sentence once.
- Although the new KB is not conceptually identical to the old KB, it will be satisfiable if the old KB was.
- This rule states that for a new constant symbol c , one can deduce $P(c)$ from the formula given in the form of $\exists x P(x)$.

- The only constraint with this rule is that c must be a new word for which $P(c)$ is true.
- It's written like this:

$$\frac{\exists x P(x)}{P(c)}$$

Example:

1

From the given sentence: $\exists x \text{Crown}(x) \wedge \text{OnHead}(x, \text{John})$,

So we can infer: $\text{Crown}(K) \wedge \text{OnHead}(K, \text{John})$, as long as K does not appear in the knowledge base.

- The above used K is a constant symbol, which is known as **Skolem constant**.
- The Existential instantiation is a special case of **Skolemization process**.

4. Existential introduction

- An existential generalization is a valid inference rule in first-order logic that is also known as an existential introduction.
- This rule argues that if some element c in the universe of discourse has the property P , we can infer that something in the universe has the attribute P .
- It's written like this:

$$\frac{P(c)}{\exists x P(x)}$$

- **Example:** Let's say that, "Priyanka got good marks in English."
"Therefore, someone got good marks in English."

Generalized Modus Ponens Rule:

In FOL, we use a single inference rule called Generalized Modus Ponens for the inference process. It's a modified form of Modus ponens.

" P implies Q , and P is declared to be true, hence Q must be true," summarizes Generalized Modus Ponens.

Modus Ponens states that for atomic phrases pi, pi', q . Where there is a substitution θ such that $SUBST(\theta, pi') = SUBST(\theta, pi)$, it can be represented as:

$$\frac{p1', p2', \dots, pn', (p1 \wedge p2 \wedge \dots \wedge pn \Rightarrow q)}{SUBST(\theta, q)}$$

Example: We will use this rule for Kings are evil, so we will find some x such that x is king, and x is greedy so we can infer that x is evil.

Propositional vs. First-Order Inference

1. Propositional Inference (also called Sentential Logic)

Propositional logic deals with **propositions** or **sentences**, which are declarative statements that are either true or false. It doesn't concern itself with the internal structure of propositions but rather with their relationships and how they can be combined.

- **Basic Components:** Propositions (e.g., P, Q, RP, Q, RP, Q, R) and logical connectives (AND, OR, NOT, IMPLIES, etc.).
- **Inference:** In propositional logic, the process of inference is typically based on a set of rules that manipulate whole propositions. Common rules of inference include Modus Ponens, Modus Tollens, Disjunctive Syllogism, etc.

Example of an inference:

- Premise 1: $P \rightarrow Q$ (If P, then Q)
- Premise 2: P (P is true)
- Conclusion: Q (Therefore, Q is true)
- **Limitations:** Propositional logic cannot express more complex relationships, like "all humans are mortal" or "some animals are mammals," because it only operates on atomic propositions without internal structure.

2. First-Order Inference (also called Predicate Logic)

First-order logic is a more powerful and expressive system. It includes not only propositions but also **quantifiers** (such as "for all" or "there exists") and **predicates** that describe properties or relations between objects. It allows you to make inferences about individual objects, classes of objects, and relationships between them.

- **Basic Components:** Predicates (e.g., $\text{Human}(x)$, $\text{Mortal}(x)$), quantifiers (e.g., $\forall x$ for "for all", $\exists x$ for "there exists"), variables (e.g., x, y), and logical connectives (AND, OR, NOT, IMPLIES, etc.).
- **Inference:** First-order inference involves reasoning about statements that can include variables and quantifiers. For example, you can infer statements like "If everyone who is a human is mortal, and Socrates is a human, then Socrates is mortal."

Example of an inference:

- Premise 1: $\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$ (For all x , if x is a human, then x is mortal)
- Premise 2: $\text{Human}(\text{Socrates})$ (Socrates is a human)
- Conclusion: $\text{Mortal}(\text{Socrates})$ (Therefore, Socrates is mortal)
- **Advantages:** First-order logic can express a much wider range of concepts, such as relationships, properties, and quantification over objects. It can also model real-world phenomena more accurately (e.g., "Every dog has a tail," or "There exists a person who is a scientist").
- **Complexity:** First-order inference is more complex than propositional inference because it involves reasoning over variables, predicates, and quantifiers. It typically requires more sophisticated techniques (like unification, resolution, or using logical entailment) to handle the logical relationships between predicates.

Comparison

Feature	Propositional Logic (PL)	First-Order Logic (FOL)
Objects	Deals only with entire propositions	Deals with individual objects and their properties or relations
Quantifiers	None	Uses quantifiers ($\forall$ for all, $\exists$ exists)
Predicates/Functions	None	Uses predicates and functions (e.g., $\text{Human}(x)$)
Expressiveness	Limited (no internal structure in propositions)	More expressive (can describe relations, properties, and quantified statements)
Inference Rules	Based on logical connectives	Based on logical connectives, quantifiers, and predicates
Complexity	Simpler, easier to compute	More complex, requires handling of variables and quantifiers

Unification and First-Order Inference

1. Unification in First-Order Logic (FOL)

Unification is a process used to find a substitution that makes two logical expressions (terms or formulas) identical. It is a key operation in logic-based systems like Prolog and plays a central role in automated reasoning, theorem proving, and logic programming.

Key Concepts:

- **Term:** A variable or constant (e.g., x , a , $f(x, y)$).
- **Substitution:** A mapping of variables to terms (e.g., $\{x \rightarrow a, y \rightarrow f(b)\}$).
- **Most General Unifier (MGU):** The simplest substitution that makes two terms or formulas identical.

Unification Process:

- **Goal:** Given two terms t_1 and t_2 , find a substitution σ such that $\sigma(t_1) = \sigma(t_2)$.
- **Example 1:** Unifying $f(x, a)$ and $f(b, c)$:
 - The unifier is $\{x \rightarrow b, a \rightarrow c\}$.
- **Example 2:** Unifying $f(x, y)$ and $f(a, a)$:
 - The unifier is $\{x \rightarrow a, y \rightarrow a\}$.

Unification Algorithm:

1. Base Case: If both terms are identical, return the identity substitution (i.e., no change needed).
2. Recursive Case:

- If the terms are compound (e.g., function symbols), recursively unify their arguments.
- If one term is a variable, substitute it with the other term, provided it doesn't lead to circular definitions (i.e., avoid situations like $x \rightarrow f(x)x \rightarrow f(x)x \rightarrow f(x)x$).

Occurs Check (optional):

- The occurs check is used to prevent a variable from being unified with a term that contains that same variable. For example, unifying xxx with $f(x)f(x)f(x)$ would create a circular dependency, leading to an infinite loop.

Example:

Unify $p(x, f(x))p(x, f(x))p(x, f(x))$ with $p(a, f(b))p(a, f(b))p(a, f(b))$:

- Start with the first arguments: xxx with aaa (i.e., $x \rightarrow ax \rightarrow a$).
- Now unify $f(a)f(a)f(a)$ with $f(b)f(b)f(b)$: This gives $a \rightarrow ba \rightarrow b$.
- The final unifier is $\{x \rightarrow a, a \rightarrow b\} \setminus \{x \rightarrow a, a \rightarrow b\} \setminus \{x \rightarrow a, a \rightarrow b\}$.

2. First-Order Inference

First-Order Logic (FOL) is an extension of propositional logic that allows reasoning with quantified variables. First-order inference involves drawing conclusions from premises expressed in FOL.

Key Elements of First-Order Logic:

- **Quantifiers:**
 - Universal Quantifier ($\forall$): "For all" — applies to all instances of a variable.
 - Existential Quantifier ($\exists$): "There exists" — applies to at least one instance of a variable.
- Predicates: Logical statements or properties about objects (e.g., $P(x)P(x)P(x)$, meaning "x is a person").
- Terms: Represent objects or values (e.g., aaa , bbb , or $f(x)f(x)f(x)$).
- Atomic Formula: Basic logical expression (e.g., $P(x)P(x)P(x)$, $R(x,y)R(x,y)R(x,y)$).

Inference Rules in First-Order Logic:

1. Universal Instantiation (UI):

- From $\forall x P(x) \forall x W, P(x) \forall x P(x)$, infer $P(a)P(a)P(a)$ for any specific aaa .

2. Existential Generalization (EG):

- From $P(a)P(a)P(a)$, infer $\exists x P(x) \exists x \neg P(x), P(x) \exists x P(x)$.

3. Modus Ponens:

- If $P \rightarrow Q$ and P , infer Q .

4. Universal Generalization (UG):

- If a statement $P(x)$ holds for an arbitrary x , conclude $\forall x P(x)$.

5. Existential Instantiation (EI):

- If $\exists x P(x)$, infer $P(a)$, where a is a new constant not previously used in the proof.

Resolution in First-Order Logic:

- Resolution is a powerful inference rule used for automated theorem proving in FOL.
- It combines two clauses with complementary literals (e.g., $P(x)$ and $\neg P(a)$) to derive new conclusions.
- The resolution procedure is complete for FOL, meaning it can derive any logically valid conclusion if it exists.

Example of First-Order Inference:

1. Premises:

- $\forall x (Human(x) \rightarrow Mortal(x))$ (All humans are mortal).
- $Human(Socrates)$ (Socrates is human).

2. Steps:

- Apply Universal Instantiation to the first premise:
 $Human(Socrates) \rightarrow Mortal(Socrates)$
- Use Modus Ponens on $Human(Socrates)$ and $Human(Socrates) \rightarrow Mortal(Socrates)$ to conclude $Mortal(Socrates)$.

Automated Reasoning and Theorem Proving:

- Forward Chaining: Start with known facts and apply inference rules to derive new facts.
- Backward Chaining: Start with a goal and work backward to see if it can be derived from known facts.

Applications of First-Order Inference:

- Expert Systems: Inference engines use FOL for reasoning about knowledge.

- Automated Theorem Proving: Proving theorems using formal logic.
- Artificial Intelligence: Problem-solving in AI systems often uses FOL to model and reason about knowledge.

3. Combining Unification with First-Order Inference:

- Unification is often a precursor to many first-order inference techniques like resolution.
- When trying to prove a goal or derive conclusions, unification helps to match terms and variables, ensuring that the inference process is logically valid.

Example (Resolution with Unification):

- Given two clauses:
 - $P(x) \vee Q(x)P(x) \vee Q(x)$
 - $\neg P(a) \vee R(a)\neg P(a) \vee R(a)\neg P(a) \vee R(a)$
- Unify $P(x)P(x)P(x)$ and $\neg P(a)\neg P(a)\neg P(a)$: The unifier is $x \rightarrow ax$ to $ax \rightarrow a$.
- Resolve $P(a)P(a)P(a)$ with $\neg P(a)\neg P(a)\neg P(a)$, leaving $Q(a) \vee R(a)Q(a) \vee R(a)Q(a) \vee R(a)$ as the resulting clause.

Forward Chaining and Backward Chaining

In artificial intelligence, forward and backward chaining is one of the important topics, but before understanding forward and backward chaining lets first understand that from where these two terms came.

Inference engine

The inference engine is the component of the intelligent system in artificial intelligence, which applies logical rules to the knowledge base to infer new information from known facts. The first inference engine was part of the expert system. Inference engine commonly proceeds in two modes, which are:

- Forward chaining
- Backward chaining

Horn Clause and Definite clause

Horn clause and definite clause are the forms of sentences, which enables knowledge base to use a more restricted and efficient inference algorithm. Logical inference algorithms use forward and backward chaining approaches, which require KB in the form of the first-order definite clause.

Definite clause: A clause which is a disjunction of literals with exactly one positive literal is known as a definite clause or strict horn clause.

Horn clause: A clause which is a disjunction of literals with at most one positive literal is known as horn clause. Hence all the definite clauses are horn clauses.

Example: $(\neg p \vee \neg q \vee k)$. It has only one positive literal k . It is equivalent to $p \wedge q \rightarrow k$.

A. Forward Chaining

Forward chaining is also known as a forward deduction or forward reasoning method when using an inference engine. Forward chaining is a form of reasoning which start with atomic sentences in the knowledge base and applies inference rules (Modus Ponens) in the forward direction to extract more data until a goal is reached.

The Forward-chaining algorithm starts from known facts, triggers all rules whose premises are satisfied, and add their conclusion to the known facts. This process repeats until the problem is solved.

Properties of Forward-Chaining

- It is a down-up approach, as it moves from bottom to top.
- It is a process of making a conclusion based on known facts or data, by starting from the initial state and reaches the goal state.
- Forward-chaining approach is also called as data-driven as we reach to the goal using available data.
- Forward -chaining approach is commonly used in the expert system, such as CLIPS, business, and production rule systems.

Consider the following famous example which we will use in both approaches:

Example

"As per the law, it is a crime for an American to sell weapons to hostile nations. Country A, an enemy of America, has some missiles, and all the missiles were sold to it by Robert, who is an American citizen."

Prove that "Robert is criminal."

To solve the above problem, first, we will convert all the above facts into first-order definite clauses, and then we will use a forward-chaining algorithm to reach the goal.

Facts Conversion into FOL

- It is a crime for an American to sell weapons to hostile nations. (Let's say p, q, and r are variables)
$$\text{American}(p) \wedge \text{weapon}(q) \wedge \text{sells}(p, q, r) \wedge \text{hostile}(r) \rightarrow \text{Criminal}(p) \quad \dots(1)$$
- Country A has some missiles. $\exists p \text{ Owns}(A, p) \wedge \text{Missile}(p)$. It can be written in two definite clauses by using Existential Instantiation, introducing new Constant T1. $\text{Owns}(A, T1)$ $\dots(2)$
 $\text{Missile}(T1) \quad \dots(3)$
- All of the missiles were sold to country A by Robert.
$$\forall p \text{ Missiles}(p) \wedge \text{Owns}(A, p) \rightarrow \text{Sells}(\text{Robert}, p, A) \quad \dots(4)$$
- Missiles are weapons.
$$\text{Missile}(p) \rightarrow \text{Weapons}(p) \quad \dots(5)$$
- Enemy of America is known as hostile.
$$\text{Enemy}(p, \text{America}) \rightarrow \text{Hostile}(p) \quad \dots(6)$$
- Country A is an enemy of America.
$$\text{Enemy}(A, \text{America}) \quad \dots(7)$$
- Robert is American
$$\text{American}(\text{Robert}). \quad \dots(8)$$

Forward chaining proof

Step-1

In the first step we will start with the known facts and will choose the sentences which do not have implications, such as: $\text{American}(\text{Robert})$, $\text{Enemy}(A, \text{America})$, $\text{Owns}(A, T1)$, and $\text{Missile}(T1)$. All these facts will be represented as below.

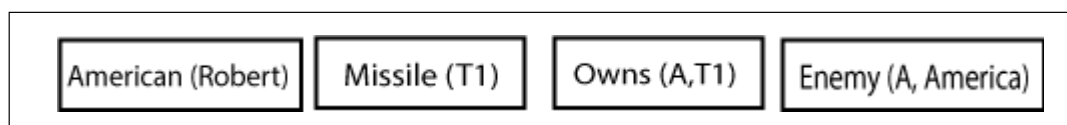


Figure 1

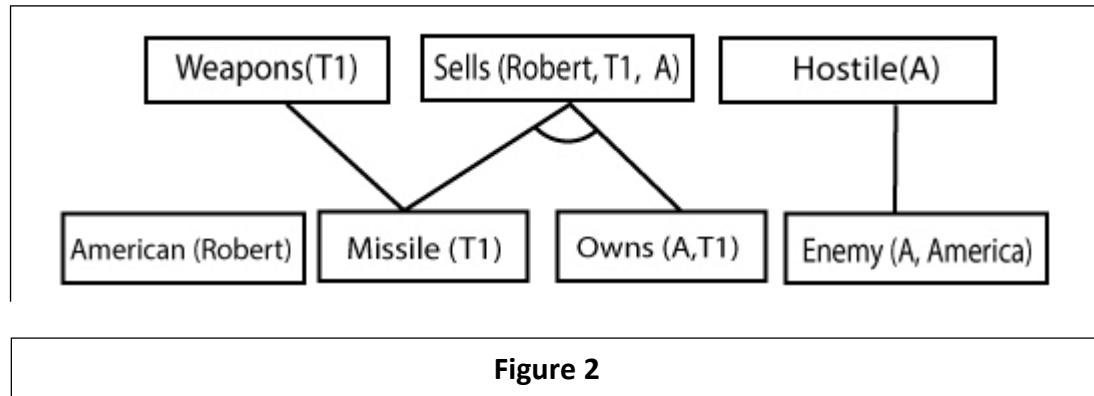
Step-2

At the second step, we will see those facts which infer from available facts and with satisfied premises.

Rule-(1) does not satisfy premises, so it will not be added in the first iteration. Rule-(2) and (3) are already added.

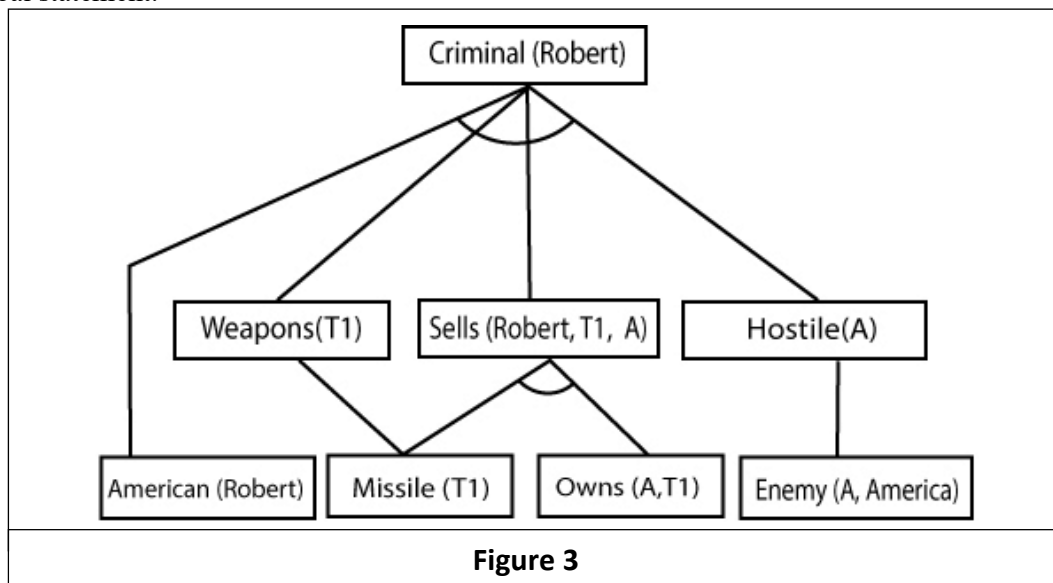
Rule-(4) satisfy with the substitution $\{p/T1\}$, so $\text{Sells}(\text{Robert}, T1, A)$ is added, which infers from the conjunction of Rule (2) and (3).

Rule-(6) is satisfied with the substitution (p/A) , so $\text{Hostile}(A)$ is added and which infers from Rule-(7).



Step-3

At step-3, as we can check Rule-(1) is satisfied with the substitution $\{p/\text{Robert}, q/T1, r/A\}$, so we can add Criminal (Robert) which infers all the available facts. And hence we reached our goal statement.



Hence it is proved that Robert is Criminal using forward chaining approach.

B. Backward Chaining

Backward-chaining is also known as a backward deduction or backward reasoning method when using an inference engine. A backward chaining algorithm is a form of reasoning, which starts with the goal and works backward, chaining through rules to find known facts that support the goal.

Properties of backward chaining

- It is known as a top-down approach.
- Backward-chaining is based on modus ponens inference rule.
- In backward chaining, the goal is broken into sub-goal or sub-goals to prove the facts true.
- It is called a goal-driven approach, as a list of goals decides which rules are selected and used.
- Backward -chaining algorithm is used in game theory, automated theorem proving tools, inference engines, proof assistants, and various AI applications.
- The backward-chaining method mostly used a depth-first search strategy for proof.

Example

In backward-chaining, we will use the same above example, and will rewrite all the rules.

- $\text{American}(p) \wedge \text{weapon}(q) \wedge \text{sells}(p, q, r) \wedge \text{hostile}(r) \rightarrow \text{Criminal}(p)$... (1)
- $\text{Owns}(A, T1)$... (2)
- $\text{Missile}(T1)$
- $?p \text{ Missiles}(p) \wedge \text{Owns}(A, p) \rightarrow \text{Sells}(\text{Robert}, p, A)$... (4)
- $\text{Missile}(p) \rightarrow \text{Weapons}(p)$... (5)
- $\text{Enemy}(p, \text{America}) \rightarrow \text{Hostile}(p)$... (6)
- $\text{Enemy}(A, \text{America})$... (7)
- $\text{American}(\text{Robert})$ (8)

Backward-Chaining proof

In Backward chaining, we will start with our goal predicate, which is $\text{Criminal}(\text{Robert})$, and then infer further rules.

Step-1

At the first step, we will take the goal fact. And from the goal fact, we will infer other facts, and at last, we will prove those facts true. So our goal fact is "Robert is Criminal," so following is the predicate of it.

Criminal (Robert)

Step-2

At the second step, we will infer other facts from goal fact which satisfies the rules. So as we can see in Rule-1, the goal predicate $\text{Criminal}(\text{Robert})$ is present with substitution $\{\text{Robert}/p\}$. So we will add all the conjunctive facts below the first level and will replace p with Robert. Here we can see $\text{American}(\text{Robert})$ is a fact, so it is proved here.

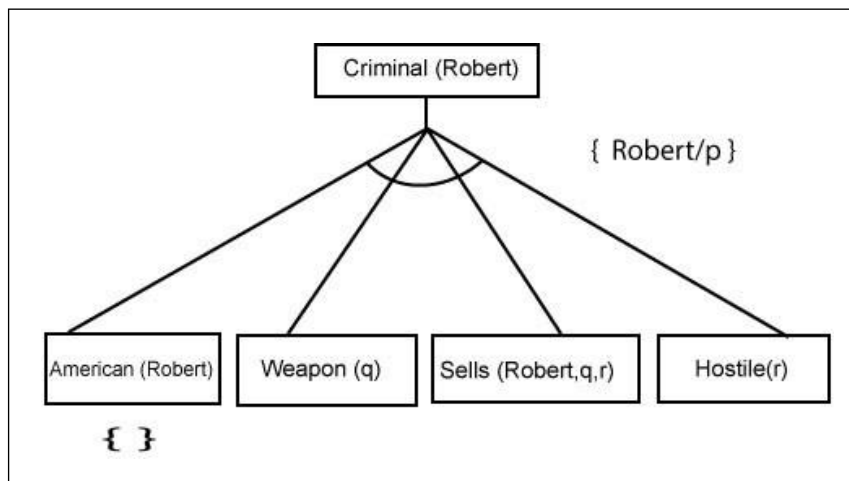


Figure 4

Step-3

At step-3, we will extract further fact Missile(q) which infer from Weapon(q), as it satisfies Rule-(5). Weapon (q) is also true with the substitution of a constant T1 at q.

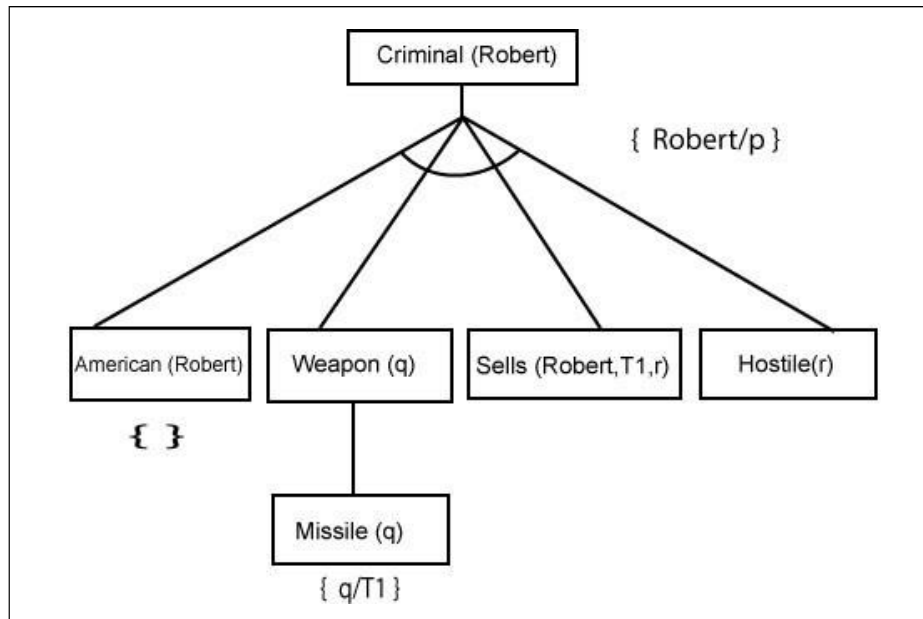


Figure 5

Step-4

At step-4, we can infer facts Missile(T1) and Owns(A, T1) from Sells(Robert, T1, r) which satisfies the Rule- 4, with the substitution of A in place of r. So these two statements are proved here.

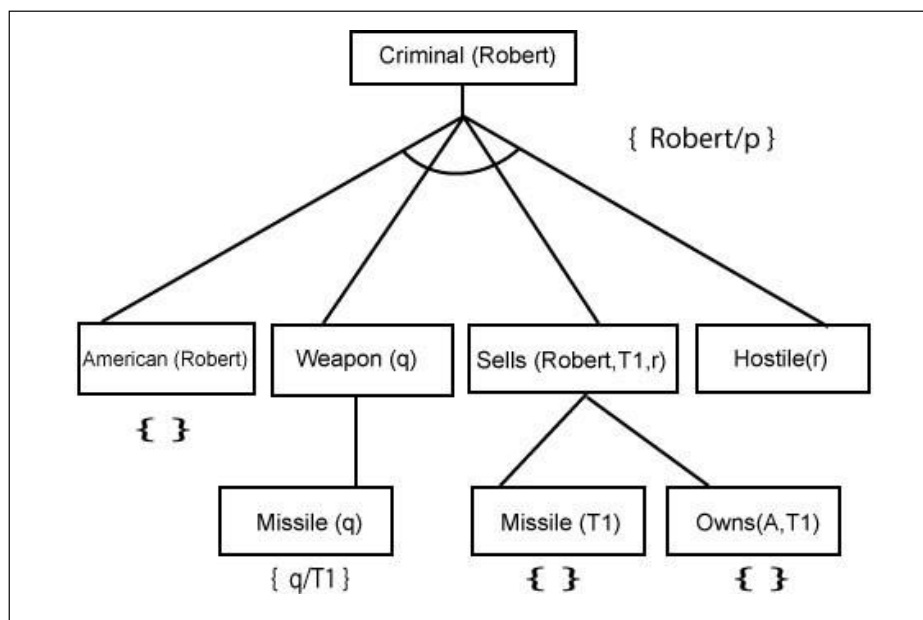


Figure 6

Step-5

At step-5, we can infer the fact $\text{Enemy}(A, \text{America})$ from $\text{Hostile}(A)$ which satisfies Rule- 6. And hence all the statements are proved true using backward chaining.

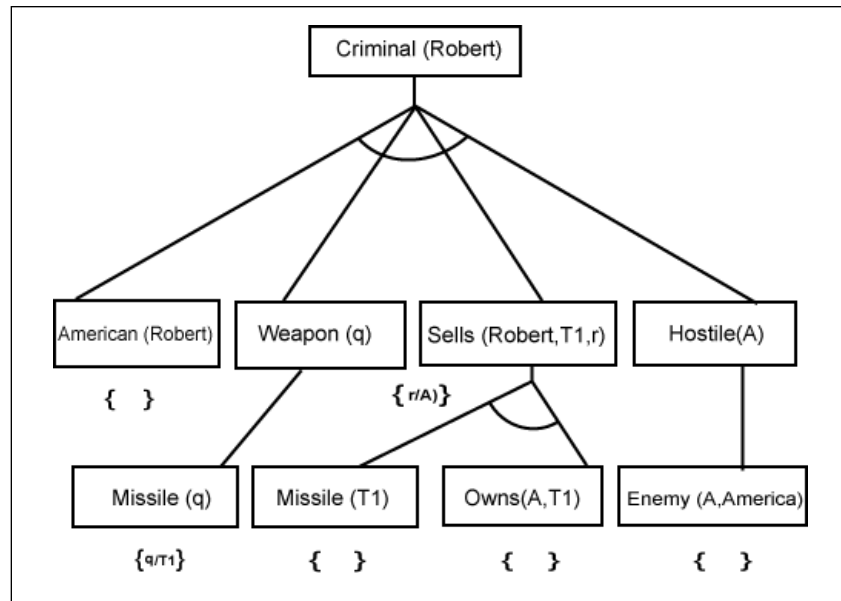


Figure 7

Suppose you have a production system with the FOUR rules: R1: IF A AND C then F R2: IF A AND E, THEN G R3: IF B, THEN E R4: R3: IF G, THEN D and you have four initial facts: A, B, C, D. PROVE A&B TRUE THEN D IS TRUE. Explain what is meant by “forward chaining”, and show explicitly how it can be used in this case to determine new facts.

3.5 RESOLUTION IN FOL

Resolution

Resolution is a theorem proving technique that proceeds by building refutation proofs, i.e., proofs by contradictions. It was invented by a Mathematician John Alan Robinson in the year 1965.

Resolution is used, if there are various statements are given, and we need to prove a conclusion of those statements. Unification is a key concept in proofs by resolutions. Resolution is a single inference rule which can efficiently operate on the conjunctive normal form or clausal form.

Clause: Disjunction of literals (an atomic sentence) is called a clause. It is also known as a unit clause.

Conjunctive Normal Form: A sentence represented as a conjunction of clauses is said to be conjunctive normal form or CNF.

Comparison

S. No.	Forward Chaining	Backward Chaining
1.	Forward chaining starts from known facts and applies inference rule to extract more data unit it reaches to the goal.	Backward chaining starts from the goal and works backward through inference rules to find the required facts that support the goal.
2.	It is a bottom-up approach	It is a top-down approach
3.	Forward chaining is known as data-driven inference technique as we reach to the goal using the available data.	Backward chaining is known as goal-driven technique as we start from the goal and divide into sub-goal to extract the facts.
4.	Forward chaining reasoning applies a breadth-first search strategy.	Backward chaining reasoning applies a depth-first search strategy.
5.	Forward chaining tests for all the available rules	Backward chaining only tests for few required rules.

Comparison

6.	Forward chaining is suitable for the planning, monitoring, control, and interpretation application.	Backward chaining is suitable for diagnostic, prescription, and debugging application.
7.	Forward chaining can generate an infinite number of possible conclusions.	Backward chaining generates a finite number of possible conclusions.
8.	It operates in the forward direction.	It operates in the backward direction.
9.	Forward chaining is aimed for any conclusion.	Backward chaining is only aimed for the required data.

Probabilistic reasoning

Uncertainty:

Till now, it is understood about knowledge representation using first-order logic and propositional logic with certainty, which means we were sure about the predicates. With this knowledge representation, we might write $A \rightarrow B$, which means if A is true then B is true, but consider a situation where we are not sure about whether A is true or not then we cannot express this statement, this situation is called uncertainty.

So to represent uncertain knowledge, where we are not sure about the predicates, we need uncertain reasoning or probabilistic reasoning.

Causes of uncertainty:

Following are some leading causes of uncertainty to occur in the real world.

- Information occurred from unreliable sources.
- Experimental Errors
- Equipment fault
- Temperature variation
- Climate change.

Probabilistic reasoning:

Probabilistic reasoning is a way of knowledge representation where we apply the concept of probability to indicate the uncertainty in knowledge. In probabilistic reasoning, we combine probability theory with logic to handle the uncertainty.

We use probability in probabilistic reasoning because it provides a way to handle the uncertainty that is the result of someone's laziness and ignorance.

In the real world, there are lots of scenarios, where the certainty of something is not confirmed, such as "It will rain today," "behavior of someone for some situations," "A match between two teams or two players." These are probable sentences for which we can assume that it will happen but not sure about it, so here we use probabilistic reasoning.

Need of probabilistic reasoning in AI:

- When there are unpredictable outcomes.
- When specifications or possibilities of predicates becomes too large to handle.
- When an unknown error occurs during an experiment.

In probabilistic reasoning, there are two ways to solve problems with uncertain knowledge:

- Bayes' rule
- Bayesian Statistics

As probabilistic reasoning uses probability and related terms, so before understanding probabilistic reasoning, let's understand some common terms:

Probability: Probability can be defined as a chance that an uncertain event will occur. It is the numerical measure of the likelihood that an event will occur. The value of probability always remains between 0 and 1 that represent ideal uncertainties.

$0 \leq P(A) \leq 1$, where $P(A)$ is the probability of an event A. $P(A) = 0$, indicates total uncertainty in an event A.

$P(A) = 1$, indicates total certainty in an event A.

We can find the probability of an uncertain event by using the below formula.

$$\text{Probability of occurrence} = \frac{\text{Number of desired outcomes}}{\text{Total number of outcomes}}$$

- $P(\neg A)$ = probability of a not happening event.
- $P(\neg A) + P(A) = 1$.

Event: Each possible outcome of a variable is called an event.

Sample space: The collection of all possible events is called sample space.

Random variables: Random variables are used to represent the events and objects in the real world.

Prior probability: The prior probability of an event is probability computed before observing new information.

Posterior Probability: The probability that is calculated after all evidence or information has taken into account. It is a combination of prior probability and new information.

Conditional probability:

Conditional probability is a probability of occurring an event when another event has already happened. Let's suppose, we want to calculate the event A when event B has already occurred, "the probability of A under the conditions of B", it can be written as:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

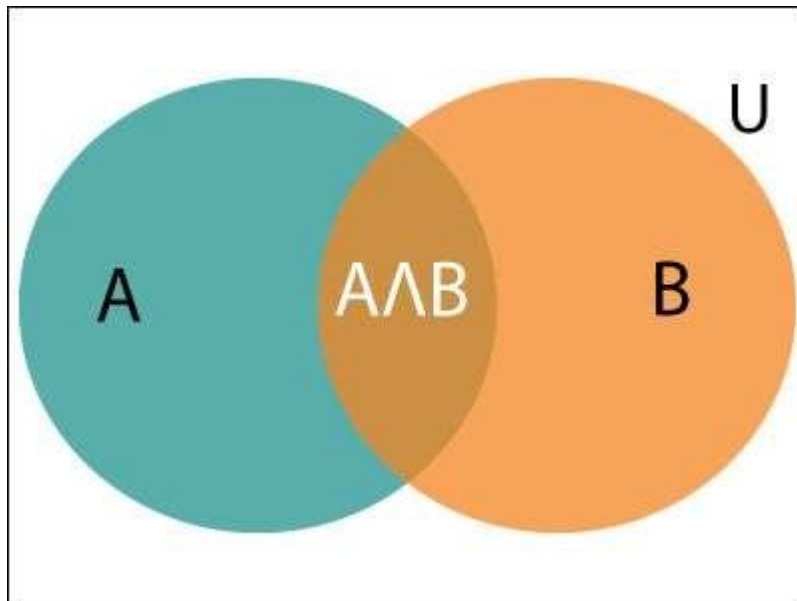
Where $P(A \cap B)$ = Joint probability of A and B

$P(B)$ = Marginal probability of B.

If the probability of A is given and we need to find the probability of B, then it will be given as:

$$P(B|A) = \frac{P(A \cap B)}{P(A)}$$

It can be explained by using the below Venn diagram, where B is occurred event, so sample space will be reduced to set B, and now we can only calculate event A when event B is already occurred by dividing the probability of $P(A \cap B)$ by $P(B)$.



Example:

In a class, there are 70% of the students who like English and 40% of the students who like English and mathematics, and then what is the percent of students those who like English also like mathematics?

Solution:

Let, A is an event that a student likes Mathematics B is an event that a student likes English.

$$P(A|B) = \frac{P(A \cap B)}{P(B)} = \frac{0.4}{0.7} = 57\%$$

Hence, 57% are the students who like English also like Mathematics.

Bayes' theorem:

Bayes' theorem is also known as Bayes' rule, Bayes' law, or Bayesian reasoning, which determines the probability of an event with uncertain knowledge.

In probability theory, it relates the conditional probability and marginal probabilities of two random events.

Bayes' theorem was named after the British mathematician Thomas Bayes. The Bayesian inference is an application of Bayes' theorem, which is fundamental to Bayesian statistics.

It is a way to calculate the value of $P(B|A)$ with the knowledge of $P(A|B)$.

Bayes' theorem allows updating the probability prediction of an event by observing new information of the real world.

Example: If cancer corresponds to one's age then by using Bayes' theorem, we can determine the probability of cancer more accurately with the help of age.

Bayes' theorem can be derived using product rule and conditional probability of event A with known event B:

As from product rule we can write:

$$P(A \wedge B) = P(A|B) P(B) \text{ or}$$

Similarly, the probability of event B with known event A: $P(A \wedge B) = P(B|A) P(A)$

Equating right hand side of both the equations, we will get:

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)} \quad \dots(a)$$

The above equation (a) is called as Bayes' rule or Bayes' theorem. This equation is basic of most modern AI systems for probabilistic inference.

It shows the simple relationship between joint and conditional probabilities. Here,

$P(A|B)$ is known as posterior, which we need to calculate, and it will be read as Probability of hypothesis A when we have occurred an evidence B.

$P(B|A)$ is called the likelihood, in which we consider that hypothesis is true, then we calculate the probability of evidence.

$P(A)$ is called the prior probability, probability of hypothesis before considering the evidence

$P(B)$ is called marginal probability, pure probability of an evidence.

In the equation (a), in general, we can write $P(B) = \sum_{i=1}^k P(A_i) * P(B|A_i)$, hence the Bayes' rule can be written as:

$$P(A_i|B) = \frac{P(A_i) * P(B|A_i)}{\sum_{i=1}^k P(A_i) * P(B|A_i)}$$

Where $A_1, A_2, A_3, \dots, A_n$ is a set of mutually exclusive and exhaustive events.

Applying Bayes' rule:

Bayes' rule allows us to compute the single term $P(B|A)$ in terms of $P(A|B)$, $P(B)$, and

$P(A)$. This is very useful in cases where we have a good probability of these three terms and want to determine the fourth one. Suppose we want to perceive the effect of some unknown cause, and want to compute that cause, then the Bayes' rule becomes:

$$P(\text{cause} | \text{effect}) = \frac{P(\text{effect} | \text{cause}) P(\text{cause})}{P(\text{effect})}$$

Example-1:

Question: what is the probability that a patient has diseases meningitis with a stiff neck?

Given Data:

A doctor is aware that disease meningitis causes a patient to have a stiff neck, and it occurs 80% of the time. He is also aware of some more facts, which are given as follows:

- The Known probability that a patient has meningitis disease is 1/30,000.
- The Known probability that a patient has a stiff neck is 2%.

Let a be the proposition that patient has stiff neck and b be the proposition that patient has meningitis. , so we can calculate the following as:

$$P(a|b) = 0.8 \quad P(b) =$$

$$1/30000 \quad P(a) = .02$$

$$P(b|a) = \frac{P(a|b)P(b)}{P(a)} = \frac{0.8 * (\frac{1}{30000})}{0.02} = 0.001333333.$$

Hence, we can assume that 1 patient out of 750 patients has meningitis disease with a stiff neck.

Example-2:

Question: From a standard deck of playing cards, a single card is drawn. The probability that the card is king is 4/52, then calculate posterior probability P(King|Face), which means the drawn face card is a king card.

Solution:

$$P(\text{king} | \text{face}) = \frac{P(\text{Face} | \text{king}) * P(\text{King})}{P(\text{Face})} \dots\dots(i)$$

P(king): probability that the card is King= 4/52= 1/13 P(face):

probability that a card is a face card= 3/13

P(Face|King): probability of face card when we assume it is a king = 1 Putting all values in equation (i) we will get:

$$P(\text{king} | \text{face}) = \frac{1 * (\frac{1}{13})}{(\frac{3}{13})} = 1/3, \text{ it is a probability that a face card is a king card.}$$

Application of Bayes' theorem in Artificial intelligence:

Following are some applications of Bayes' theorem:

1. It is used to calculate the next step of the robot when the already executed step is given.
2. Bayes' theorem is helpful in weather forecasting.
3. It can solve the Monty Hall problem.